

The 8-Puzzle Problem: Hill Climbing and A* Search Algorithms

AI Lab Assignment

February 3, 2026

Abstract

This document provides a comprehensive mathematical treatment of the 8-puzzle problem and its solution using Hill Climbing and A* search algorithms. We present formal definitions, algorithm pseudocode, heuristic functions, complexity analysis, and experimental results.

Contents

1	Introduction	2
1.1	Problem Statement	2
2	Heuristic Functions	2
2.1	Manhattan Distance Heuristic	2
2.2	Misplaced Tiles Heuristic	3
2.3	Dominance Relation	3
3	Hill Climbing Algorithm	3
3.1	Basic Hill Climbing	3
3.2	Hill Climbing with Sideways Moves	3
3.3	Complexity Analysis	3
3.4	Limitations	5
4	A* Search Algorithm	5
4.1	Cost Function	5
4.2	Algorithm	5
4.3	Optimality and Completeness	5
4.4	Complexity Analysis	5
5	Experimental Results	5
5.1	Test Cases	5
5.1.1	Test Case 1: Simple Problem	7
5.1.2	Test Case 2: Medium Complexity	7
5.2	Performance Comparison	7
5.3	Analysis	7

6	Mathematical Properties	8
6.1	State Space Size	8
6.2	Branching Factor	8
7	Conclusion	9
8	References	9

1 Introduction

The 8-puzzle problem is a classic problem in artificial intelligence that involves sliding numbered tiles on a 3×3 grid to reach a goal configuration. This problem serves as an excellent testbed for studying search algorithms and heuristic functions.

1.1 Problem Statement

Definition 1 (8-Puzzle State). *An 8-puzzle state S is a 3×3 matrix where each cell contains either a number from $\{1, 2, 3, 4, 5, 6, 7, 8\}$ or a blank space (denoted as 0):*

$$S = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \\ s_{31} & s_{32} & s_{33} \end{bmatrix}$$

where $s_{ij} \in \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ and each number appears exactly once.

Definition 2 (Valid Move). *A valid move m slides a tile adjacent to the blank space into the blank space. The set of valid moves from state S is:*

$$M(S) = \{m : m \in \{Up, Down, Left, Right\} \wedge \text{move } m \text{ is within bounds}\}$$

Definition 3 (State Space). *The state space $\mathcal{S}$ is the set of all reachable states from an initial state S_0 through valid moves. The state space forms a directed graph $G = (\mathcal{S}, E)$ where:*

- $\mathcal{S}$ is the set of all possible board configurations
- $E = \{(S_i, S_j) : S_j \text{ can be reached from } S_i \text{ by one valid move}\}$

2 Heuristic Functions

Heuristic functions estimate the cost from a current state to the goal state. A good heuristic function is both *admissible* (never overestimates) and *informative* (provides useful guidance).

2.1 Manhattan Distance Heuristic

Definition 4 (Manhattan Distance). *For a state S , the Manhattan distance heuristic $h_M(S)$ is the sum of the distances of each tile from its goal position:*

$$h_M(S) = \sum_{i=1}^8 |x_i - x_i^*| + |y_i - y_i^*|$$

where (x_i, y_i) is the current position of tile i and (x_i^*, y_i^*) is its goal position.

Theorem 1 (Admissibility of Manhattan Distance). *The Manhattan distance heuristic h_M is admissible for the 8-puzzle problem.*

Proof. Each tile must move at least the Manhattan distance to reach its goal position. Since each move changes the position of exactly one tile by one unit, the Manhattan distance never overestimates the minimum number of moves required. $\square$

2.2 Misplaced Tiles Heuristic

Definition 5 (Misplaced Tiles). *The misplaced tiles heuristic $h_T(S)$ counts the number of tiles not in their goal position:*

$$h_T(S) = |\{i : s_i \neq s_i^* \wedge s_i \neq 0\}|$$

where s_i is the value at position i in state S and s_i^* is the goal value.

Theorem 2 (Admissibility of Misplaced Tiles). *The misplaced tiles heuristic h_T is admissible for the 8-puzzle problem.*

Proof. Each misplaced tile requires at least one move to reach its goal position. Therefore, $h_T(S)$ never overestimates the actual cost. $\square$

2.3 Dominance Relation

Theorem 3 (Heuristic Dominance). *For all non-goal states S in the 8-puzzle:*

$$h_M(S) \geq h_T(S)$$

Thus, Manhattan distance dominates the misplaced tiles heuristic.

Proof. For any misplaced tile, its Manhattan distance from the goal is at least 1. Therefore:

$$h_M(S) = \sum_{i \in \text{misplaced}} d_M(i) \geq \sum_{i \in \text{misplaced}} 1 = h_T(S)$$

$\square$

3 Hill Climbing Algorithm

Hill Climbing is a local search algorithm that iteratively moves to neighboring states with better heuristic values.

3.1 Basic Hill Climbing

3.2 Hill Climbing with Sideways Moves

To escape plateaus, we allow a limited number of sideways moves (moves to states with equal heuristic value).

3.3 Complexity Analysis

Theorem 4 (Time Complexity of Hill Climbing). *In the worst case, Hill Climbing explores $O(b^d)$ nodes, where b is the branching factor and d is the solution depth. However, in practice, it typically explores $O(d)$ nodes for problems without local optima.*

Theorem 5 (Space Complexity). *Hill Climbing has space complexity $O(|\text{visited}|) = O(d)$ in the best case, where d is the number of steps taken.*

Algorithm 1 Hill Climbing Search

```
1: procedure HILLCLIMBING( $S_0, S_{goal}, h$ )
2:    $current \leftarrow S_0$ 
3:    $visited \leftarrow \{S_0\}$ 
4:   while  $current \neq S_{goal}$  do
5:      $neighbors \leftarrow \text{GetSuccessors}(current) \setminus visited$ 
6:     if  $neighbors = \emptyset$  then
7:       return FAILURE ▷ Stuck at local optimum
8:     end if
9:      $best \leftarrow \arg \min_{n \in neighbors} h(n)$ 
10:    if  $h(best) \geq h(current)$  then
11:      return FAILURE ▷ No improvement
12:    end if
13:     $current \leftarrow best$ 
14:     $visited \leftarrow visited \cup \{best\}$ 
15:  end while
16:  return  $current$ 
17: end procedure
```

Algorithm 2 Hill Climbing with Sideways Moves

```
1: procedure HILLCLIMBINGSIDEWAYS( $S_0, S_{goal}, h, max_{sideways}$ )
2:    $current \leftarrow S_0$ 
3:    $visited \leftarrow \{S_0\}$ 
4:    $sideways\_count \leftarrow 0$ 
5:   while  $current \neq S_{goal}$  do
6:      $neighbors \leftarrow \text{GetSuccessors}(current) \setminus visited$ 
7:     if  $neighbors = \emptyset$  then
8:       return FAILURE
9:     end if
10:     $best \leftarrow \arg \min_{n \in neighbors} h(n)$ 
11:    if  $h(best) < h(current)$  then
12:       $current \leftarrow best$ 
13:       $sideways\_count \leftarrow 0$ 
14:    else if  $h(best) = h(current)$  and  $sideways\_count < max_{sideways}$  then
15:       $current \leftarrow best$ 
16:       $sideways\_count \leftarrow sideways\_count + 1$ 
17:    else
18:      return FAILURE
19:    end if
20:     $visited \leftarrow visited \cup \{best\}$ 
21:  end while
22:  return  $current$ 
23: end procedure
```

3.4 Limitations

Hill Climbing suffers from three main problems:

1. **Local Optima:** Getting stuck at states where all neighbors have worse heuristic values
2. **Plateaus:** Flat regions where neighbors have equal heuristic values
3. **Ridges:** Sequences of local maxima difficult to navigate

4 A* Search Algorithm

A* is an informed search algorithm that combines actual cost and heuristic estimates to find optimal solutions.

4.1 Cost Function

Definition 6 (A* Evaluation Function). *For a state S , A^* uses the evaluation function:*

$$f(S) = g(S) + h(S)$$

where:

- $g(S)$ is the actual cost from the start state to S
- $h(S)$ is the heuristic estimate from S to the goal
- $f(S)$ is the estimated total cost through S

4.2 Algorithm

4.3 Optimality and Completeness

Theorem 6 (Optimality of A*). *If the heuristic function h is admissible, A^* is guaranteed to find an optimal solution.*

Theorem 7 (Completeness of A*). *A^* is complete: if a solution exists, A^* will find it.*

4.4 Complexity Analysis

Theorem 8 (Time Complexity of A*). *The time complexity of A^* is $O(b^d)$ in the worst case, where b is the branching factor and d is the solution depth. With a good heuristic, this can be reduced significantly.*

Theorem 9 (Space Complexity of A*). *A^* has space complexity $O(b^d)$ as it stores all generated nodes.*

5 Experimental Results

5.1 Test Cases

We evaluated both algorithms on several 8-puzzle instances:

Algorithm 3 A* Search

```
1: procedure ASTAR( $S_0, S_{goal}, h$ )
2:    $openSet \leftarrow$  priority queue containing  $S_0$ 
3:    $closedSet \leftarrow \emptyset$ 
4:    $g(S_0) \leftarrow 0$ 
5:    $f(S_0) \leftarrow h(S_0)$ 
6:   while  $openSet \neq \emptyset$  do
7:      $current \leftarrow$  node in  $openSet$  with minimum  $f$  value
8:     if  $current = S_{goal}$  then
9:       return RECONSTRUCTPATH( $current$ )
10:    end if
11:    Remove  $current$  from  $openSet$ 
12:    Add  $current$  to  $closedSet$ 
13:    for each  $neighbor$  in GetSuccessors( $current$ ) do
14:      if  $neighbor \in closedSet$  then
15:        continue
16:      end if
17:       $tentative\_g \leftarrow g(current) + 1$ 
18:      if  $neighbor \notin openSet$  or  $tentative\_g < g(neighbor)$  then
19:         $g(neighbor) \leftarrow tentative\_g$ 
20:         $f(neighbor) \leftarrow g(neighbor) + h(neighbor)$ 
21:         $parent(neighbor) \leftarrow current$ 
22:        if  $neighbor \notin openSet$  then
23:          Add  $neighbor$  to  $openSet$ 
24:        end if
25:      end if
26:    end for
27:  end while
28:  return FAILURE
29: end procedure
```

5.1.1 Test Case 1: Simple Problem

Initial State:

$$S_0 = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 4 & 0 \\ 7 & 6 & 5 \end{bmatrix}$$

Goal State:

$$S_{goal} = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Results:

- Hill Climbing (Manhattan): 1 move, 1 node expanded
- A* Search (Manhattan): 1 move, 1 node expanded

5.1.2 Test Case 2: Medium Complexity

Initial State:

$$S_0 = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 8 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Goal State:

$$S_{goal} = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 0 & 4 \\ 7 & 6 & 5 \end{bmatrix}$$

Results:

- Hill Climbing (Manhattan): 1 move, 1 node expanded
- A* Search (Manhattan): 1 move, 1 node expanded

5.2 Performance Comparison

Algorithm	Moves	Nodes Expanded
Hill Climbing (Manhattan)	1	1
Hill Climbing (Misplaced)	Variable	Variable
A* (Manhattan)	1	1
A* (Misplaced)	1	Variable

Table 1: Algorithm comparison on simple test cases

5.3 Analysis

Advantages of Hill Climbing:

- Low memory requirements: $O(d)$
- Fast execution for problems without local optima

- Simple implementation

Disadvantages of Hill Climbing:

- Can get stuck at local optima
- Not guaranteed to find a solution
- Not guaranteed to find optimal solution

Advantages of A*:

- Guaranteed to find optimal solution (with admissible heuristic)
- Complete: always finds solution if one exists
- Efficient with good heuristics

Disadvantages of A*:

- High memory requirements: $O(b^d)$
- Can be slow for complex problems

6 Mathematical Properties

6.1 State Space Size

Theorem 10 (8-Puzzle State Space). *The total number of possible configurations in the 8-puzzle is $9! = 362,880$. However, only half are reachable from any given initial state due to parity constraints.*

Proof. There are $9!$ ways to arrange 9 tiles (including the blank). The 8-puzzle can be proven to have two disjoint classes of states based on permutation parity, making only $9!/2 = 181,440$ states reachable from any given initial state. $\square$

6.2 Branching Factor

Lemma 1 (Average Branching Factor). *The average branching factor of the 8-puzzle is approximately 2.67.*

Proof. The blank can be in one of 9 positions:

- Corner positions (4): 2 possible moves each
- Edge positions (4): 3 possible moves each
- Center position (1): 4 possible moves

Average: $\frac{4(2)+4(3)+1(4)}{9} = \frac{24}{9} \approx 2.67$ $\square$

7 Conclusion

We have presented a comprehensive mathematical treatment of the 8-puzzle problem and two solution approaches: Hill Climbing and A* search. While Hill Climbing offers simplicity and efficiency for easy problems, A* provides optimality guarantees at the cost of higher memory usage.

The choice between algorithms depends on the problem characteristics:

- Use Hill Climbing for: simple problems, memory-constrained environments, quick approximate solutions
- Use A* for: problems requiring optimal solutions, when memory is available, guaranteed completeness

8 References

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.
3. Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97-109.